项目仓库：MDMS_Course_2026Fall

基于 MongoDB 的问卷系统第一阶段项目完成报告

姓名：庄子由

阅读入口：数据库设计 PDF • API 说明 PDF • 测试报告 PDF • AI 使用日志 PDF • 技术讲解与检查指南 PDF • README

一 项目目标与完成范围

本项目实现一个简化的在线问卷系统。注册用户可以创建、发布和关闭问卷，通过链接邀请已登录用户填写，并查看整个问卷或单道题的统计。

第一阶段已实现单选、多选、文本与数字填空，可设置必答、选择数量、文本长度、数值范围及整数限制，并支持由单选、多选和数字答案触发的可配置跳转。系统保存每次成功提交的答卷，支持匿名显示、按问卷决定是否允许重复填写，以及截止时间控制。

实现范围以初始用户需求为准。题库、题目共享、版本历史与跨问卷统计属于第二阶段，本次保留现有代码作为后续扩展基础。

二 系统设计

系统由普通网页、Flask 后端和 MongoDB 数据库组成。本次实际环境为 Python 3.12.7、Flask 3.1.3、PyMongo 4.19.0、MongoDB 8.3.11，依赖见 requirements.txt。

前端模板在 templates/，交互脚本与样式在 static/。app.py 负责路由、登录、权限、问卷状态和保存操作；logic.py 负责题目定义、答案校验、跳转与统计；db.py 负责连接、集合初始化及索引。运行和配置方法见根目录 README，接口细节见 API 说明 PDF。

注册密码使用 Werkzeug 的 scrypt 哈希保存。登录后使用签名会话 Cookie 标识当前账号，修改操作检查 CSRF 令牌。用户的管理查询同时限制问卷标识和创建者标识，避免通过直接修改 URL 访问他人的问卷与统计。

三 MongoDB 设计

数据库使用 users、surveys、responses 三个集合，所有业务数据均存储在 MongoDB。实际基础结构校验规则保存在 schema.json，新集合创建时由程序应用。

users 保存用户名、密码哈希和注册时间。surveys 保存创建者、标题、说明、发布状态、匿名及重复填写设置、截止时间和题目数组。每道题内嵌选项、校验配置及跳转规则。responses 为每次成功提交保存一份独立文档，关联问卷和填写账号，并保存规范化后的答案与时间。

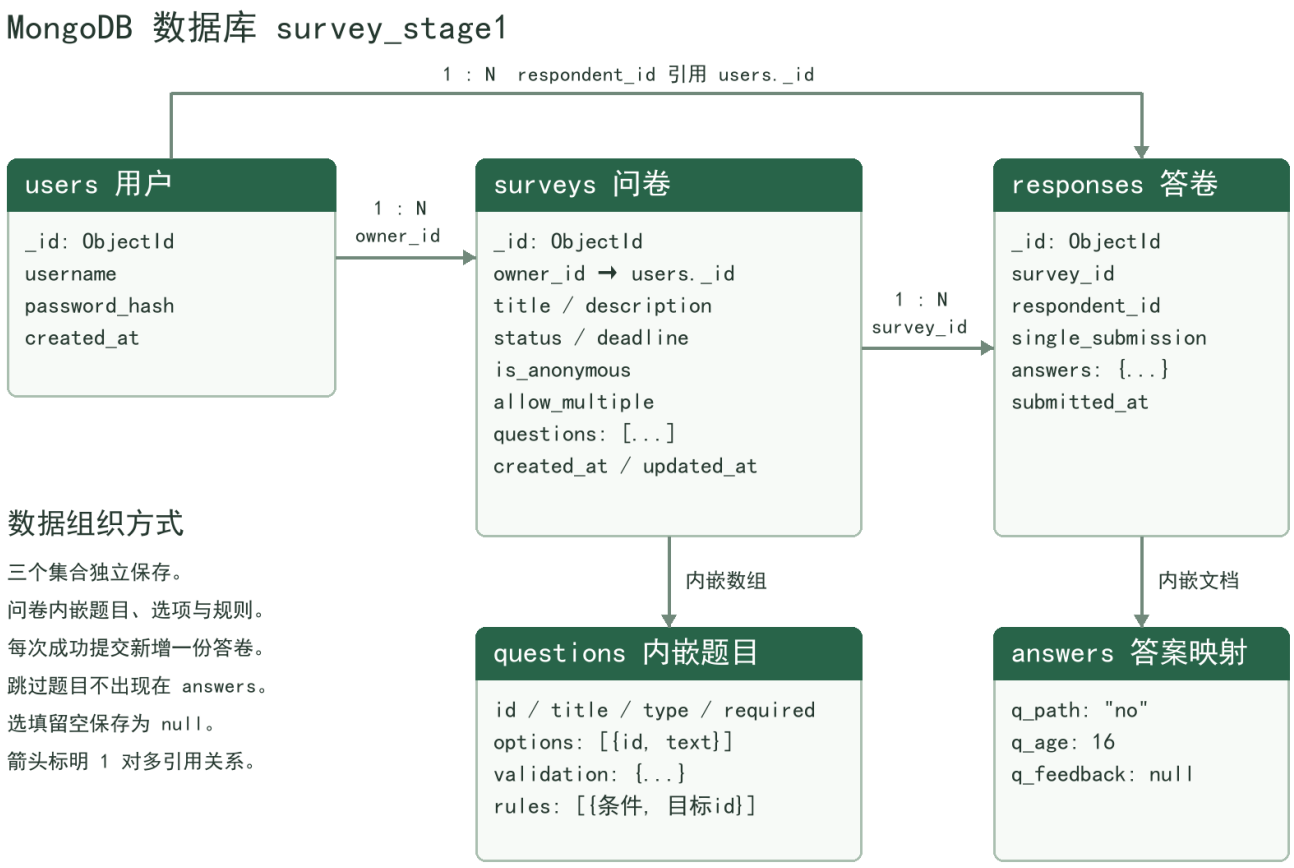
这样的划分依据是访问方式：打开问卷时通常一起使用题目、选项和规则，嵌入文档可以自然表达并一次取出；答卷持续增长，独立集合可避免问卷文档无限变大。用户独立存在，供问卷归属和登录关联使用。系统没有机械地把每个嵌套结构拆成关系表，也没有在每份答卷中复制整份问卷。

题目和选项使用稳定标识，答案与跳转引用标识，题目序号只决定展示顺序。发布后固定内容和规则，保证已收集数据仍对应原题。

除默认 _id 索引外，建立用户名唯一索引、按创建者及时间查询问卷的索引、按问卷及时间查询答卷的索引，以及禁止重复填写时生效的部分唯一索引。后者仅约束 single_submission=true 的答卷，防止同一账号并发保存多份受限答卷。该字段由后端根据问卷设置生成。

完整字段与索引说明见 数据库设计 PDF。

数据结构图



MongoDB数据结构

图中的连线表示通过 ObjectId 保存关联：一位用户可以创建多份问卷，一份问卷可以收到多份答卷。题目数组位于问卷内部，选项、校验与跳转规则又位于各题目内部；它们不是另外三个集合。

用命令查看实际数据

先按运行说明启动 MongoDB，并生成演示样例。下面在 PowerShell 连接项目数据库，后续命令均在 mongosh 中执行；这些命令只查询，不修改数据。

```
mongosh "mongodb://127.0.0.1:27017/survey_stage1"
```

确认当前数据库与集合，并用字段投影只查看账号标识、用户名和注册时间：

```
db.getName()
show collections
db.users.find({}, { _id: 1, username: 1, created_at: 1 })
```

用演示标记找到问卷，再沿同一个 _id 查看题目与答卷。这里的 demo 只是当前命令行会话中的变量。

```
var demo = db.surveys.findOne(
  { demo_tag: "stage1_walkthrough_v1" },
  { title: 1, owner_id: 1, status: 1, is_anonymous: 1,
    allow_multiple: 1, deadline: 1, questions: 1 }
)
demo
demo.questions.map(q => ({
  id: q.id, type: q.type, options: q.options,
  validation: q.validation, rules: q.rules
}))
```

```
db.responses.find(
  { survey_id: demo._id },
  { _id: 1, answers: 1, submitted_at: 1 }
).sort({ submitted_at: 1 })
```

demo 若为 null，表示当前库中还没有这份演示样例，应先完成演示初始化。答卷里的 q_feedback: null 表示到达了选填题但留空；缺少 q_fruits 表示该题被跳过。MongoDB 的 {字段: null} 查询同时匹配空值和缺失，严格区分时要再使用 $exists。

查看三个集合的实际校验器和索引：

```
printjson(db.getCollectionInfos().map(c => ({
  name: c.name, validator: c.options.validator
})))
db.users.getIndexes()
db.surveys.getIndexes()
db.responses.getIndexes()
```

最后直接用 MongoDB 聚合核算年龄平均值，独立于网页及 Python 统计函数。$match 选择这份问卷中的数值年龄，$group 计算回答数量和平均值。

```
db.responses.aggregate([
  { $match: { survey_id: demo._id, "answers.q_age": { $type: "number" } } },
  { $group: { _id: null, answered: { $sum: 1 }, average: { $avg: "$answers.q_age" } } },
  { $project: { _id: 0, answered: 1, average: 1 } }
])
```

演示脚本最初生成两份年龄为 20、16 的答卷，尚未追加填写时应得到 answered: 2、average: 18。它与浏览器检查所用的另一份问卷不同；该浏览器问卷的年龄是 20、40，平均值为 30。继续填写后，查询结果应随实际数据变化。

四 校验与跳转设计

创建问卷时，后端先检查题型、标识是否重复、数量和范围是否合理、规则选项及目标是否存在，并限制跳转只能指向后面的题目。多选“恰好 N 项”通过最少和最多都设置为 N 表达。

填写时，单选检查有效选项；多选检查选项、重复值和数量；文本按去除首尾空白后的字符数检查；数字转换为有限数值并检查范围和整数要求。选填题留空保存为 null，不检查最短长度或数值下限。

每道题的规则按顺序判断，采用第一条命中规则；没有命中则进入下一题。单选比较选项相等，多选判断包含指定选项，数字支持等于和大小比较。规则保存在问卷中，由统一函数执行，没有为某份问卷写死跳转。

网页每次前进调用逐题检查接口，后端返回规范化答案及下一题；最终提交时后端仍从第一题重新计算路径，并只校验、保存实际路径上的答案。这样，被跳过的必答题不会误报漏答，切换分支后的残留答案不会污染统计。

五 问卷状态与统计

问卷分草稿、已发布、已关闭。草稿可修改定义；发布至少需要一道有效题目；发布时用原子条件确认题目与截止时间仍是检查过的内容，防止并发编辑造成错误发布。截止或关闭后，提交接口拒绝新答卷，已有统计保留。

匿名指对问卷创建者隐藏身份，填写者仍需登录；系统内部保留账号关联以实现重复填写限制。匿名统计及明细不输出填写者账号或内部账号标识。

统计读取成功提交的答卷实时计算，不维护额外的统计副本。单选统计各选项人数和回答总数，多选统计各选项次数；填空显示有效内容，数字计算有效数值平均值。跳过和留空不计入题目回答人数。允许重复填写时，各次提交均独立计数。

六 测试结果

本次完成 23 项纯逻辑单测和 26 项真实 MongoDB 集成测试，共 49 项，均通过。测试源文件为 tests/test_logic.py 与 tests/test_app.py。覆盖作业要求的创建问卷、添加题目、跳转、校验、提交和统计，并检查权限、匿名、截止、关闭、CSRF、发布后固定内容及并发提交。

并发验证中，禁止重复的同一账号 5 个请求仅保存 1 份，其余返回 409；允许重复时 5 个请求保存 5 份独立答卷。另有一项回归模拟发布检查期间另一标签页清空草稿，确认发布被拒绝而不会产生零题已发布问卷。

浏览器还实际完成注册、四题问卷编辑、规则配置、发布、两条路径填写、错误输入提示、两份答卷提交和统计筛选。两份年龄 20、40 的回答在网页显示平均值 30，跳过题的回答人数及匿名明细正确。

测试步骤、输入、预期、实际结果及原始输出分别见 测试报告 PDF 和 测试原始输出文本。这些结果验证了列出的场景，未进行公网部署或大规模压力测试。

七 AI 使用过程与遇到的问题

本项目使用 Codex 进行需求分析、计划、代码生成、文档整理和运行验证；并行子代理分别实现业务逻辑、独立集成测试及只读审查。关键提示词、采用的代码和处理过程见 AI 使用日志 PDF。

AI 提议并实现了集合划分、题目嵌入结构、跳转与校验逻辑、部分唯一索引及前后端交互。用户确定作业范围、简化要求、推进节奏，并要求提供可以自行复核的技术说明。截至本报告生成，尚未记录用户人工修改代码或完成后续独立复核，不能将 AI 生成内容记作人工独立设计。

开发中遇到的一个实际设计问题是发布操作与并发保存之间的竞态：最初发布只根据状态更新，可能在题目被清空后继续发布。独立审查发现后，将题目定义与截止时间加入原子更新条件，并加入真实数据库回归测试。环境搭建时还处理了临时目录、外部日志配置与受限网络影响，最终通过实际连接及运行检查。

八 运行与交付

交付包含前后端源码、MongoDB schema、固定依赖、配置示例、演示数据生成脚本、测试代码与正式文档。虚拟环境、私有配置、数据库原始文件和开发临时目录不进入提交包。

运行顺序为启动 MongoDB、安装依赖并启动 Flask。可用 scripts/seed_demo.py 生成明确标注的演示账号、问卷和两份答卷，随后按 技术讲解与检查指南 PDF 从网页追踪到代码与数据库。重复运行生成脚本不覆盖已有演示数据。

仓库以根目录组织源码、测试和 README；水杉提交的项目完成报告 PDF 文件名为 2026_ECNU_PJ1_1_庄子由.pdf，仓库链接位于本文最前面。